ATENTINA × YOIZEN

Tu propio agente

Sus agentes, nuestra voz.

Cómo los bots que Yoizen ya tiene atienden y hacen llamadas, con Atentina poniendo la voz y el teléfono.

Los bots de Yoizen ya saben qué decir. Falta que atiendan el teléfono.

Hoy

WhatsApp, web chat, bots sin código, IVR y ACD.

Su orquestador decide qué responder.

Lo que falta

Voz: llamadas entrantes y salientes.

Con esos mismos bots, configurados por sus equipos.

Lo nuevo

Tu propio agente. En cada turno, Atentina le pregunta a su orquestador qué decir.

Y lo dice, con voz argentina, por teléfono y WhatsApp.

Atentina pone la voz y el teléfono. Yoizen, las respuestas.

PONE ATENTINA

- El número, las llamadas entrantes y salientes, WhatsApp y el link para llamar
- La transcripción, los turnos y las interrupciones
- La voz argentina, la grabación y el resumen por email
- Los minutos, la derivación a personas, los eventos y el registro en el panel

PONE YOIZEN

- El prompt y la personalidad de cada bot
- Las herramientas y la conexión con sus sistemas
- La memoria y el estado de la conversación
- Qué responder en cada turno

No es cambiar el modelo. Acá el agente entero es de Yoizen: Atentina no pone ni prompt ni modelo de lenguaje.

La persona habla. Atentina transcribe, pregunta a su endpoint y dice la respuesta.



Un pedido por cada turno de la persona, en el formato de la API de OpenAI. Cada pedazo de texto que devuelve el endpoint pasa a voz apenas llega, sin esperar la respuesta completa.

Toda la conversación, en cada turno.

HTTP

```
POST https://agente.yoizen.com/v1/chat/completions
Authorization: Bearer <la key de la conexión>
X-Atentina-Conversation-Id: 6f1c2a9e-…
X-Atentina-Channel: voice
X-Atentina-Mode: entrante
X-Atentina-From: +5493515551234
X-Atentina-To: +541152630861
X-Atentina-Agent: ventas
X-Atentina-Turn: 2

{"model": "ventas", "stream": true,
 "messages": [
  {"role": "assistant", "content": "Hola, soy Sofía. ¿Qué necesitás?"},
  {"role": "user", "content": "¿Tienen turno el jueves?"},
  {"role": "assistant", "content": "Sí, a las diez o a las cuatro."},
  {"role": "user", "content": "A las cuatro. Gracias."}
 ],
 "tools": [{"type": "function", "function": {"name": "end_call", …}}]}
```

- **Sin estado de su lado.** El historial que mandamos es el que vale.
- **Los datos de la llamada, en headers:** quién llama, a qué número, canal, agente, turno.
- **Solo campos estándar.** Si lo reenvían tal cual a su modelo, funciona.
- **Varios bots, un endpoint:** cada agente manda su `model`.

En pedazos. Cada uno pasa a voz apenas llega.

SSE

```
data: {"choices":[{"delta":{"content":"Perfecto, te espero el jueves "}}]}

data: {"choices":[{"delta":{"content":"a las cuatro. Que andes bien."}}]}

data: {"choices":[{"delta":{"tool_calls":[{"function":
        {"name":"end_call",
         "arguments":"{\"outcome\": \"turno_dado\"}"}}]}}]}

data: [DONE]
```

MIENTRAS LLEGA EL SEGUNDO PEDAZO

«Perfecto, te espero el jueves…»

ya está sonando en el teléfono.

- **Streaming obligatorio.** Una respuesta entera en JSON cuenta como error.
- `delta.content` se dice; el razonamiento se descarta.
- Sin texto ni herramientas, cuenta como error.

Dos herramientas. Las manda y las ejecuta Atentina.

`end_call`

Corta la llamada

Cuando termina de decir la despedida de ese mensaje.

Con `outcome` elige el **resultado de la conversación**, entre los definidos en el agente.

Ejemplo: `turno_dado` o `sin_turno`. Queda en el panel, en el webhook y en el resumen por email.

`transfer_call`

Pasa la llamada a una persona

A un **área del equipo** (`area`), de las que tiene el agente.

La persona escucha música mientras tanto; si nadie atiende, el agente retoma.

Llega solo si el agente tiene áreas de derivación configuradas.

Si su bot las ignora, la llamada termina cuando corta la persona o al llegar a la duración máxima.

Veinte líneas: su prompt, su modelo, en streaming.

PYTHON · FASTAPI

```python
app = FastAPI()
llm = AsyncOpenAI()   # su LLM: cualquiera compatible con OpenAI

@app.post("/v1/chat/completions")
async def chat(request: Request, x_atentina_from: str = Header("")):
    body = await request.json()
    messages = [{"role": "system",
                 "content": f"{PROMPT}\nLlama: {x_atentina_from}."},
                *body["messages"]]
    stream = await llm.chat.completions.create(
        model="gpt-4.1-mini", messages=messages,
        tools=body.get("tools"), stream=True)

    async def events():
        async for chunk in stream:
            yield f"data: {chunk.model_dump_json(exclude_none=True)}\n\n"
        yield "data: [DONE]\n\n"

    return StreamingResponse(events(), media_type="text/event-stream")
```

- **Agrega su prompt** y los datos de la llamada (headers).
- **Le pasa la conversación y las herramientas** a su modelo. Si llama a `end_call`, Atentina corta.
- **Devuelve el stream tal cual.**
- El resto es de Yoizen: su orquestador, sus flujos, sus datos.

Tres pasos en el panel. O por API y MCP.

1

La conexión

Configuración › Modelos › Nueva conexión, tipo **OpenAI compatible**: la URL de su endpoint y la API key que espera.

La key queda cifrada y no se vuelve a mostrar.

2

Probar el endpoint

Le manda un turno como el de una llamada y mide cuánto tarda el primer texto.

`Tu agente respondió · streaming · «Hola, …»`

3

El agente

Prender **Que conteste tu propio agente**: endpoint, modelo, mensaje de error y qué hacer si falla.

Nombre, voz, apertura, áreas, grabación y número: como en cualquier agente.

Por API: `POST /clients/{id}/llm-connections`, `…/test` y `POST /agents`, con una key con alcance `calls`.

Por MCP, desde su agente de IA: `create_llm_connection`, `test_llm_connection`, `create_agent`.

Lo que más pesa: el tiempo hasta el primer texto.



Se nota en la llamada

Su modelo, sus herramientas y su infraestructura.

Desde Azure EE.UU. se suma la red: a medir en el piloto.

Más de un segundo

Se siente lento, con silencios.

Se ve por turno en el panel (Detalle técnico) y en Probar el endpoint.

Texto primero, herramientas después

«Dame un segundo que me fijo.»

Una frase corta antes de la consulta lenta.

Si su endpoint falla, la llamada no se queda muda.

Falla

Error, vacío o sin texto a tiempo

La espera del primer texto se configura (`first_token_timeout_ms`).

El agente dice

El mensaje configurado

«Disculpá, se me cortó. ¿Me lo repetís?»

Varios seguidos

Corta o pasa a su equipo

`max_errors` y `on_errors` : `hang_up` o `transfer:<área>` .

Un turno bueno vuelve el contador a cero.

Si se corta después de empezar a hablar, queda lo dicho y no cuenta como error.

Un endpoint y un caso real. El resto lo ponemos nosotros.

1

Su endpoint

Uno de sus bots detrás de `/chat/completions`, https público, con streaming.

2

Conexión y prueba

Juntos, en el panel. Medimos el primer texto desde su infraestructura.

3

Llamadas de prueba

Con un número nuestro, por teléfono y WhatsApp, hasta ajustar prompt y tiempos.

4

Piloto con un caso real

Un cliente, un flujo. Calidad, capacidad y costos antes de ofrecerlo.

Qué ponemos nosotros: número argentino, el agente configurado, panel con transcripciones y acompañamiento técnico. Incluido en el plan Red, a medida. La conexión SIP con su Asterisk sigue en evaluación. Guía: `atentina.com.ar/desarrolladores/docs/tu-propio-agente`

Atentina

Sus agentes, nuestra voz y nuestro teléfono.

Contacto: Laureano Arcanio

0800-220-1233

atentina.com.ar · hola@atentina.com.ar

WhatsApp +54 9 351 700-2592